

Ch4

Week 4

5 GBM(Gradient Boosting Machine)

GBM의 개요 및 실습

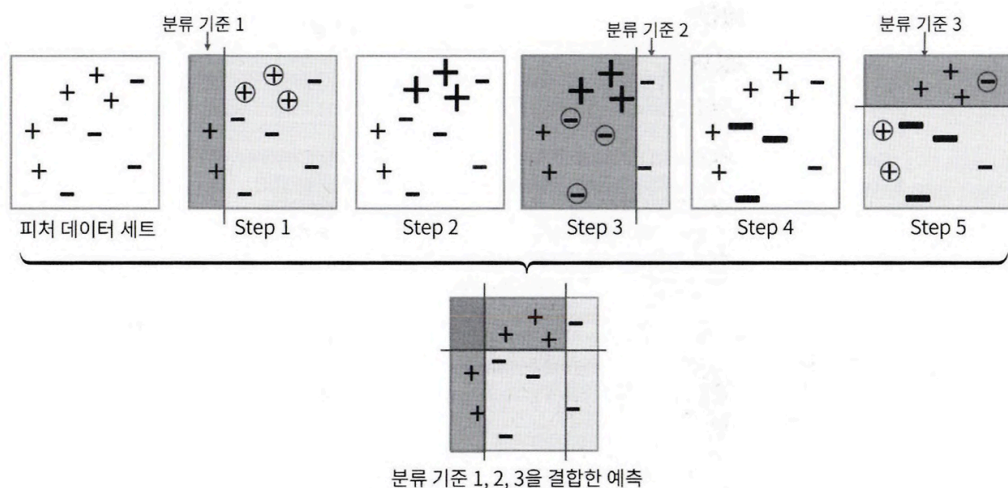
부스팅 알고리즘

- 약한 학습기(weak learner) 여러 개를 순차적 학습-예측
- 잘못 예측한 데이터에 가중치 부여를 통해 오류 개선하며 학습

부스팅의 대표적 구현

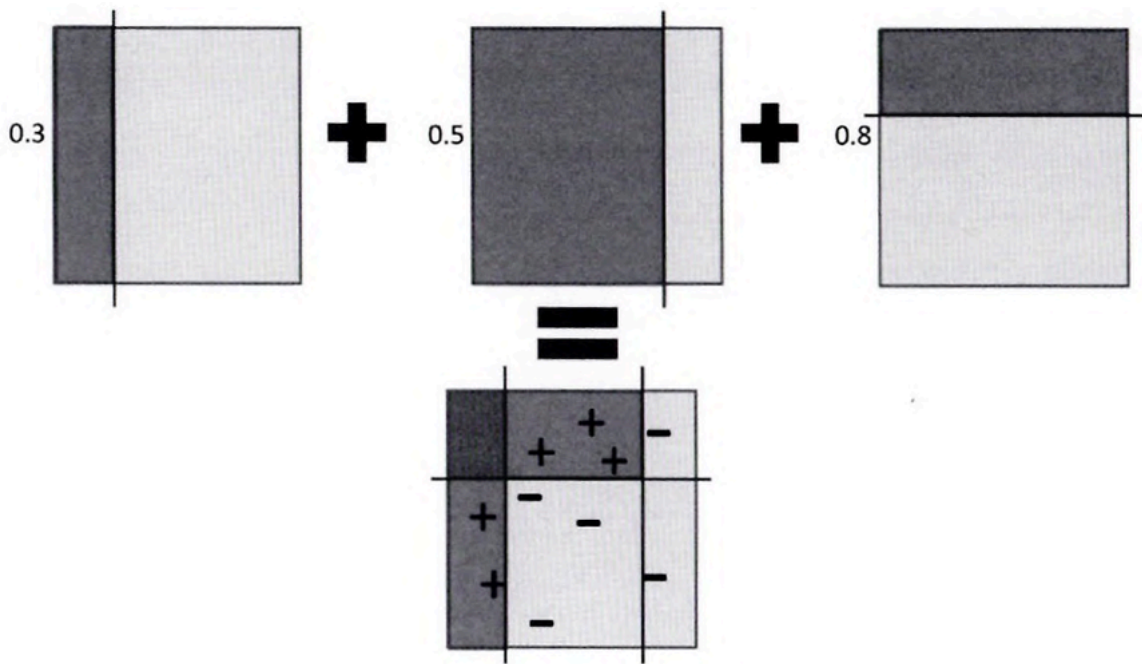
- AdaBoost(Adaptive Boosting)
 - 오류 데이터에 가중치 부여하며 부스팅하는 대표적 알고리즘
- 그래디언트 부스트

AdaBoost



- + / - 로 된 feature 데이터 세트
- Step1

- 첫 번째 약한 학습기가 분류 기준 1로 +와 -를 분류
- 동그라미 표시된 + 데이터는 + 데이터가 잘못 분류된 오류 데이터
- Step2
 - 오류 데이터에 대한 가중치 부여
 - 가중치 부여된 오류 + 데이터는 다음 약한 학습기가 잘 분류할 수 있게 크기 커짐
- Step3
 - 두 번째 약한 학습기가 분류 기준2로 +와 -를 분류
 - 동그라미 표시된 - 데이터는 잘못된 분류
- Step4
 - 잘못 분류된 - 오류 데이터에 대해 더 큰 가중치 부여
- Step5
 - 세 번째 약한 학습기가 분류 기준 3으로 +와 - 분류, 오류 데이터 찾을
 - 에이다부스트는 이렇게 약한 학습기가 순차적으로 오류 값에 대해 가중치 부여한 예측 결정 기준 모두 결합하여 예측 수행
- 맨 아래는 1, 2, 3 번째 약한 학습기를 모두 결합한 결과 예측
 - ⇒ 개별 약한 학습기보다 훨씬 정확도 높아짐
- 개별 약한 학습기는 각각 가중치를 부여해 결합



- e.g) 첫 번째 학습기 0.3, 두 번째 학습기 0.5, 세 번째 학습기 0.8
- 모두 결합하여 예측 수행

GBM(Gradient Boost Machine)

- 에이다부스트와 유사한, 가중치 업데이트를 경사 하강법(Gradient Descent)를 이용하는 것이 큰 차이
- 오류 값 = (실제 값 - 예측값)
- 분류의 실제 결괏값을 y , feature를 $x_1, x_2, x_3, \dots$, 이 feature에 기반한 예측 함수를 $F(x)$ 라 하면,
 - 오류식 $h(x) = y - F(x)$
- 이 오류식 $h(x) = y - F(x)$ 를 최소화하는 방향으로 반복적 가중치 값 업데이트가 경사 하강법(Gradient Descent)
 - $\Rightarrow$ 반복 수행을 통해 오류를 최소화할 수 있도록 가중치의 업데이트 값을 도출하는 기법
- GBM은 CART기반의 다른 알고리즘과 마찬가지로 분류, 회귀 다 가능

사이킷런 GBM 기반 분류

```

from sklearn.ensemble import GradientBoostingClassifier
import time
import warnings
warnings.filterwarnings('ignore')

X_train, X_test, y_train, y_test = get_human_dataset()

start_time=time.time()

gb_clf = GradientBoostingClassifier(random_state=0)
gb_clf.fit(X_train, y_train)
gb_pred = gb_clf.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)

print('GBM 정확도: {0:.4f}'.format(gb_accuracy))
print('GBM 수행 시간:{0:.1f}초'.format(time.time()-start_time))

```

GBM 정확도: 0.9379
GBM 수행 시간:1243.3초

- 사이킷런은 GradientBoostingClassifier 클래스 제공
- 기본 하이퍼 파라미터만으로 93.79% 예측 정확도
- 일반적으로 GBM이 랜덤 포레스트보다 예측 성능 조금 나은 경우 많음
- 약한 학습기의 순차적 예측 오류 보정 통해 학습 수행
 - 멀티 CPU 코어 시스템 사용하더라도 병렬 처리 안 돼서 대용량 데이터의 경우 학습 시간 오래 걸림

GBM 하이퍼 파라미터 소개

loss	- 경사 하강법에서 사용할 비용 함수 지정 - 특별한 이유 없다면 기본값인 'deviance' 적용
learning_rate	- GBM이 학습 진행할 때마다 적용하는 학습률 - Weak learner가 순차적으로 오류 값 보정해 나가는 데 적용하는 계수 - 0~1 사이 값 지정 가능 - 기본값 0.1 - 너무 작은 값 적용하면 업데이트 되는 값 작아져 최소 오류 값 찾아 예측 성능 높아질 가능성 높음 - But 많은 weak learner는 순차적 반복이 필요해서 수행 시간 오래 걸림 - 또 너무 작게 설정하면 모든 weak learner의 반복이 완료돼도 최소 오류 값을 못 찾을 수 있음

	- 반대로 큰 값 적용 시 최소 오류 값 찾지 못 하고 그냥 지나쳐버려 예측 성능 떨어질 가능성 높음, 수행은 빠름 - 따라서 learning rate는 n_estimators와 상호 보완적으로 조합해 사용 - learning rate를 작게 하고 n_estimators를 크게 하면 더 이상 성능 좋아지지 않는 한계점까지 예측 성능이 조금씩 좋아질 수 있음 - But 수행 시간이 너무 오래 걸리는 단점, 예측 성능 역시 현격히 좋아지지는 않음
n_estimators	- weak learner의 개수 - weak learner가 순차적으로 오류 보정하므로 개수 많을수록 예측 성능 일정 수준까지 좋아질 수 있음 - But 개수 많을수록 수행 시간 오래 걸림 - 기본값 100
subsample	- weak learner가 학습에 사용하는 데이터 샘플링 비율 - 기본값 1 → 전체 학습 데이터를 기반으로 학습한다는 의미 - 0.5면 학습 데이터의 50% - 과적합이 염려되는 경우 1보다 작은 값으로 설정

- GBM은 과적합에도 강한 뛰어난 예측 성능 but 수행 시간 오래 걸림

6 XGBoost(eXtra Gradient Boost)

XGBoost 개요

- 트리 기반의 앙상블 학습에서 가장 각광 받는 알고리즘 중 하나
- 일반적으로 다른 머신러닝보다 뛰어난 예측 성능 가짐
- GBM 기반이지만 느린 수행 시간, 과적합 규제(regularization) 부재 등의 문제 해결
- 병렬 CPU 환경에서 병렬 학습이 가능
 - 기존 GBM보다 빠르게 학습 완료 가능
- XGBoost의 주요 장점

뛰어난 예측 성능	- 일반적으로 분류와 회귀 영역서 뛰어난 예측 성능 발휘
GBM 대비 빠른 수행 시간	- 일반적인 GBM은 순차적으로 Weak learner가 가중치 증감하는 방법으로 학습 → 전반적 속도 느림 - But XGBoost는 병렬 수행 및 다양한 기능으로 GBM에 비해 빠른 수행

	성능 보장 - 아쉽게도 다른 ML 알고리즘(랜덤 포레스트 등)에 비해 빠른 건아님
과적합 규제 (Regularization)	- 표준 GBM의 경우 과적합 규제 기능이 없으나 XGBoost는 자체에 과적합 규제 기능으로 과적합에 더 강한 내구성 가짐
Tree pruning(나무 가지 치기)	- 일반적으로 GBM은 분할 시 부정 손실 발생하면 분할 멈춤 ⇒ 이 방식도 자칫 지나친 분할을 발생시킬 수 있음 - 다른 GBM처럼 XGBoost도 max_depth 파라미터로 분할 깊이 조정하기도 하지만, tree pruning으로 더 이상 긍정 이득 없는 분할 가지치기 해서 분할 수를 더 줄이는 추가적 장점 가짐
자체 내장된 교차 검증	- 반복 수행마다 내부적으로 학습 데이터 세트와 평가 데이터 세트에 대한 교차 검증 수행해 최적화된 반복 수행 횟수 가질 수 있음 - 지정된 반복 횟수 아니라 교차 검증 통해 평가 데이터 세트의 평가 값이 최적화 되면 반복 중간에 멈추는 조기 중단 기능도 있음
결손값 자체 처리	- XGBoost는 결손값을 자체 처리할 수 있는 기능 가짐

- 핵심 라이브러리가 C/C++로 작성됨
- 파이썬 패키지 xgboost 제공
- 파이썬 네이티브 XGBoost는 고유의 API와 하이퍼 파라미터 이용
 - 크게 다르진 않지만 몇 가지 주의 사항 有

XGBoost 설치하기

- pip 이용하여 쉽게 설치 가능

```
(base) C:\Users\inseo> pip install xgboost==1.5.0
Collecting xgboost==1.5.0
  Downloading xgboost-1.5.0-py3-none-win_amd64.whl.metadata (1.7 kB)
Requirement already satisfied: numpy in c:\users\inseo\anaconda3\lib\site-packages (from xgboost==1.5.0) (1.26.4)
Requirement already satisfied: scipy in c:\users\inseo\anaconda3\lib\site-packages (from xgboost==1.5.0) (1.13.1)
Downloading xgboost-1.5.0-py3-none-win_amd64.whl (106.6 MB)
106.6/106.6 MB 28.4 MB/s eta 0:00:00
Installing collected packages: xgboost
Successfully installed xgboost-1.5.0
```

파이썬 래퍼 XGBoost 하이퍼 파라미터

- 일반 파라미터
 - 일반적으로 실행 시 스레드 개수나 silent 모드 등 선택을 위한 파라미터

- 디폴트 파라미터 값을 바꾸는 경우는 거의 없음

- **booster**: gbtree(tree based model) 또는 gblinear(linear model) 선택. 디폴트는 gbtree입니다.
- **silent**: 디폴트는 0이며, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정합니다.
- **nthread**: CPU의 실행 스레드 개수를 조정하며, 디폴트는 CPU의 전체 스레드를 다 사용하는 것입니다. 멀티 코어/스레드 CPU 시스템에서 전체 CPU를 사용하지 않고 일부 CPU만 사용해 ML 애플리케이션을 구동하는 경우에 변경합니다.

- 부스터 파라미터

- 트리 최적화, 부스팅, regularization 등과 관련 파라미터 등을 지칭

- 대부분의 하이퍼 파라미터

- **eta [default=0.3, alias: learning_rate]**: GBM의 학습률(learning rate)과 같은 파라미터입니다. 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 파이썬 래퍼 기반의 xgboost를 이용할 경우 디폴트는 0.3. 사이킷런 래퍼 클래스를 이용할 경우 eta는 learning_rate 파라미터로 대체되며, 디폴트는 0.1입니다. 보통은 0.01 ~ 0.2 사이의 값을 선호합니다.
- **num_boost_rounds**: GBM의 n_estimators와 같은 파라미터입니다.
- **min_child_weight[default=1]**: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 weight 총합. min_child_weight이 클수록 분할을 자제합니다. 과적합을 조절하기 위해 사용됩니다.
- **gamma [default=0, alias: min_split_loss]**: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값입니다. 해당 값보다 큰 손실(loss)이 감소된 경우에 리프 노드를 분리합니다. 값이 클수록 과적합 감소 효과가 있습니다.
- **max_depth[default=6]**: 트리 기반 알고리즘의 max_depth와 같습니다. 0을 지정하면 깊이에 제한이 없습니다. Max_depth가 높으면 특정 피쳐 조건에 특화되어 룰 조건이 만들어지므로 과적합 가능성이 높아지며 보통은 3~10 사이의 값을 적용합니다.
- **sub_sample[default=1]**: GBM의 subsample과 동일합니다. 트리가 커져서 과적합되는 것을 제어하기 위해 데이터를 샘플링하는 비율을 지정합니다. sub_sample=0.5로 지정하면 전체 데이터의 절반을 트리를 생성하는 데 사용합니다. 0에서 1 사이의 값이 가능하나 일반적으로 0.5 ~ 1 사이의 값을 사용합니다.
- **colsample_bytree[default=1]**: GBM의 max_features와 유사합니다. 트리 생성에 필요한 피쳐(칼럼)를 임의로 샘플링하는 데 사용됩니다. 매우 많은 피쳐가 있는 경우 과적합을 조정하는 데 적용합니다.
- **lambda [default=1, alias: reg_lambda]**: L2 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- **alpha [default=0, alias: reg_alpha]**: L1 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- **scale_pos_weight [default=1]**: 특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터입니다.

- 학습 태스크 파라미터

- 학습 수행 시의 객체 함수, 평가를 위한 지표 등을 설정하는 파라미터

- **objective**: 최솟값을 가져야 할 손실 함수를 정의합니다. XGBoost는 많은 유형의 손실함수를 사용할 수 있습니다. 주로 사용되는 손실함수는 이진 분류인지 다중 분류인지에 따라 달라집니다.
- **binary:logistic**: 이진 분류일 때 적용합니다.
- **multi:softmax**: 다중 분류일 때 적용합니다. 손실함수가 multi:softmax일 경우에는 레이블 클래스의 개수인 num_class 파라미터를 지정해야 합니다.
- **multi:softprob**: multi:softmax와 유사하나 개별 레이블 클래스의 해당되는 예측 확률을 반환합니다.
- **eval_metric**: 검증에 사용되는 함수를 정의합니다. 기본값은 회귀인 경우는 rmse, 분류일 경우에는 error입니다. 다음은 eval_metric의 값 유형입니다.
 - **rmse**: Root Mean Square Error
 - **mae**: Mean Absolute Error
 - **logloss**: Negative log-likelihood
 - **error**: Binary classification error rate (0.5 threshold)
 - **merror**: Multiclass classification error rate
 - **mlogloss**: Multiclass logloss
 - **auc**: Area under the curve

• 과적합 문제가 심각하다면 적용 고려 사항

- eta 값을 낮춥니다(0.01 ~ 0.1). eta 값을 낮출 경우 num_round(또는 n_estimators)는 반대로 높여줘야 합니다.
- max_depth 값을 낮춥니다.
- min_child_weight 값을 높입니다.
- gamma 값을 높입니다.
- 또한 subsample과 colsample_bytree를 조정하는 것도 트리가 너무 복잡하게 생성되는 것을 막아 과적합 문제에 도움이 될 수 있습니다.

파이썬 래퍼 XGBoost 적용 - 위스콘신 유방암 예측

xgboost

- 자체적으로 교차 검증, 성능 평가, feature 중요도 등의 시각화(plotting) 기능 가짐
- 조기 중단 기능 有
 - num_rounds로 지정한 부스팅 반복 횟수에 도달하지 않아도 더 이상 예측 오류 개선 안 되면 중지
 - → 수행 시간 개선 기능

위스콘신 유방암 데이터 세트

- 종양의 크기, 모양 등 다양한 속성값을 기반으로 악성 종양(malignant)인지 양성 종양(benign)인지 분류한 데이터 세트
- 양성 종양이 비교적 성장 속도 느리고 전이 안 됨
- 악성 종양은 주위 조직에 침입하면서 빠르게 성장하고 신체 각 부위에 확산되거나 전이 되어 생명 위협
- feature에 따라 악성인지 양성인지를 XGBoost를 이용해 예측

```
import xgboost as xgb
from xgboost import plot_importance
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

dataset = load_breast_cancer()
features = dataset.data
labels = dataset.target

cancer_df = pd.DataFrame(data=features, columns=dataset.feature_names)
cancer_df['target'] = labels
cancer_df.head(3)
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry
0	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.6	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601
1	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.8	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750
2	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613

- 종양의 크기와 모양에 관련된 많은 속성이 숫자형 값으로 돼 있음
- 타깃 레이블 값의 종류는 악성인 malignant가 0, 양성인 benign이 1값으로 됨
- 레이블 값의 분포 확인

```
print(dataset.target_names)
print(cancer_df['target'].value_counts())
```

```
['malignant' 'benign']
target
1    357
0    212
Name: count, dtype: int64
```

- 1값인 benign이 357개, 0값인 malignant가 212개로 구성

- 데이터 세트의 80%를 학습용으로, 20%를 테스트용으로 추출 후 80%의 학습용 데이터에서 90%를 최종 학습용, 10%를 검증용으로 분할

```
X_features = cancer_df.iloc[:, :-1]
y_label = cancer_df.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(X_features, y_label,
                                                    test_size=0.2, random_state=156 )

X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train,
                                            test_size=0.1, random_state=156 )

print(X_train.shape , X_test.shape)
print(X_tr.shape, X_val.shape)

(455, 30) (114, 30)
(409, 30) (46, 30)
```

- 검증용 별도 분할 이유는 XGBoost가 제공하는 기능인 검증 성능 평가, 조기 중단 수행하기 위하여
- 전체 569개 데이터 세트 중 최종 학습용 409개, 검증용 46개, 테스트용 114개 추출

파이썬 래퍼 XGBoost

사이킷런과의 차이

- XGBoost만의 전용 데이터 객체인 DMatrix사용
- Numpy 또는 Pandas로 된 학습용, 검증, 테스트용 데이터 세트 모두 전용 데이터 객체인 DMatrix로 생성하여 모델에 입력해야 함
- 넘파이 외에도 DataFrame과 Series 기반으로도 DMatrix를 생성할 수 있음
- DMatrix의 주요 입력 파라미터
 - data
 - feature 데이터 세트
 - label
 - 분류일 때 레이블 데이터 세트
 - 회귀일 때 숫자형인 종속값 데이터 세트

- DataFrame 기반의 학습 데이터 세트와 테스트 데이터 세트 DMatrix로 변환

```
dtr = xgb.DMatrix(data=X_tr, label=y_tr)
dval = xgb.DMatrix(data=X_val, label=y_val)
dtest = xgb.DMatrix(data=X_test, label=y_test)
```

- xgboost를 이용해 학습하기 전 XGBoost의 하이퍼 파라미터 설정
 - 주로 딕셔너리 형태로 입력

```
params = { 'max_depth':3,
           'eta': 0.05,
           'objective':'binary:logistic',
           'eval_metric':'logloss'
         }

num_rounds = 400
```

- max_depth 3
- 학습률 eta 0.1
 - XGBClassifier 사용하면 eta가 아니라 learning_rate
- 예제 데이터 0 또는 1 이진 분류이므로 목적함수는 이진 로지스틱
- 오류 함수의 평가 성능 지표는 logloss.
- num_rounds(부스팅 반복 횟수) 400

지정된 하이퍼 파라미터로 XGBoost 모델 학습

- 파이썬 래퍼 XGBoost는 하이퍼 파라미터를 xgboost 모듈의 train() 함수에 파라미터로 전달
 - 사이킷런은 Estimator의 생성자를 하이퍼 파라미터로 전달함(차이점)

- 학습 시 XGBoost는 조기 중단 기능 제공
 - 수행 속도 개선하기 위해
 - 수행 성능 개선하기 위해 더 이상 지표 개선 없을 경우 num_boost_round 횟수 다 채우지 않고 중간에 반복 중단
- 조기 중단 성능 평가는 주로 별도 검증 데이터 세트 이용
- XGBoost는 학습 반복 시마다 검증 데이터 세트 이용
 - 성능 평가 기능 제공
- 조기 중단은 xgboost의 train() 함수에 early_stopping_rounds 파라미터 입력하여 설정
- early_stopping_rounds 파라미터를 설정해 조기 중단 숏애하기 위해서는 반드시 평가용 데이터 세트 지정과 eval_metric을 함께 설정해야 함
 - xgboost는 반복마다 지정된 평가용 데이터 세트에서 eval_metric의 지정된 평가 지표로 예측 오류 측정함
 - 평가용 데이터 세트는 학습과 평가용 데이터 세트를 명기하는 개별 튜플 가지는 리스트 형태로 설정
 - 만약 dtr이 학습용, dval이 평가용이라면 [(dtr,'train'),(dval,'eval')]
 - 학습용 DMatrix는 'train'으로, 평가용 DMatrix는 'eval'로 개별 튜플에서 명기하여 설정
 - eval_metric은 평가 세트에 적용할 성능 평가 방법
 - 분류일 경우 주로 'error'(분류 우로), 'logloss' 적용

xgboost 모듈의 train() 함수 호출, 학습 수행

- 평가용 데이터 세트 설정은 학습용 DMatrix인 dtr과 검증용 DMatrix은 dval로 설정
- train() 함수의 evals 인자값으로 입력
- eval_metric은 위에서 params 딕셔너리로 지정됨
- Xgboost 학습 반복 시마다 evals에 설정된 데이터 세트에 대해 평가 지표 결과 출력됨
- train()은 학습 완료된 모델 객체 반환


```
[11] eval_list = [(dtr, 'train'), (dval, 'eval')]
```

```
xgb_model = xgb.train(params = params , dtrain=dtr , num_boost_round=num_rounds , #  
                      early_stopping_rounds=50, evals=eval_list )
```



```
[193] train-logloss:0.01146 eval-logloss:0.23722  
[194] train-logloss:0.01141 eval-logloss:0.23715  
[195] train-logloss:0.01136 eval-logloss:0.23661  
[196] train-logloss:0.01128 eval-logloss:0.23673  
[197] train-logloss:0.01126 eval-logloss:0.23651  
[198] train-logloss:0.01120 eval-logloss:0.23628  
[199] train-logloss:0.01113 eval-logloss:0.23641  
[200] train-logloss:0.01108 eval-logloss:0.23554  
[201] train-logloss:0.01103 eval-logloss:0.23533  
[202] train-logloss:0.01097 eval-logloss:0.23543  
[203] train-logloss:0.01091 eval-logloss:0.23546  
[204] train-logloss:0.01086 eval-logloss:0.23588  
[205] train-logloss:0.01079 eval-logloss:0.23600  
[206] train-logloss:0.01074 eval-logloss:0.23578  
[207] train-logloss:0.01069 eval-logloss:0.23597  
[208] train-logloss:0.01064 eval-logloss:0.23601  
[209] train-logloss:0.01058 eval-logloss:0.23613  
[210] train-logloss:0.01054 eval-logloss:0.23592  
[211] train-logloss:0.01047 eval-logloss:0.23605  
[212] train-logloss:0.01045 eval-logloss:0.23586  
[213] train-logloss:0.01041 eval-logloss:0.23626  
[214] train-logloss:0.01036 eval-logloss:0.23646  
[215] train-logloss:0.01031 eval-logloss:0.23649  
[216] train-logloss:0.01026 eval-logloss:0.23697  
[217] train-logloss:0.01024 eval-logloss:0.23679  
[218] train-logloss:0.01019 eval-logloss:0.23689  
[219] train-logloss:0.01014 eval-logloss:0.23693  
[220] train-logloss:0.01011 eval-logloss:0.23714  
[221] train-logloss:0.01006 eval-logloss:0.23732  
[222] train-logloss:0.01004 eval-logloss:0.23714  
[223] train-logloss:0.01001 eval-logloss:0.23736  
[224] train-logloss:0.00999 eval-logloss:0.23711  
[225] train-logloss:0.00997 eval-logloss:0.23703  
[226] train-logloss:0.00994 eval-logloss:0.23757
```

- train()으로 학습 수행하면서 반복 시마다 train-logloss가 지속적으로 감소함
- num_boost_round를 400회로 설정했음에도 학습은 400번 반복 안 함
 - 0부터 시작하여 176번째 반복에서 완료함
 - 126번째 반복에서 176번까지 early_stopping_rounds로 지정된 50회 동안 logloss 값은 더 향상되지 않았기에 반복 멈춤

xgboost 이용하여 모델 학습 완료되면 테스트 데이터 세트에 예측 수행

```

pred_probs = xgb_model.predict(dtest)
print('predict( ) 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨')
print(np.round(pred_probs[:10],3))

preds = [ 1 if x > 0.5 else 0 for x in pred_probs ]
print('예측값 10개만 표시:',preds[:10])

```

```

predict( ) 수행 결과값을 10개만 표시, 예측 확률 값으로 표시됨
[0.938 0.004 0.75  0.049 0.98  1.      0.999 0.999 0.998 0.001]
예측값 10개만 표시: [1, 0, 1, 0, 1, 1, 1, 1, 1, 0]

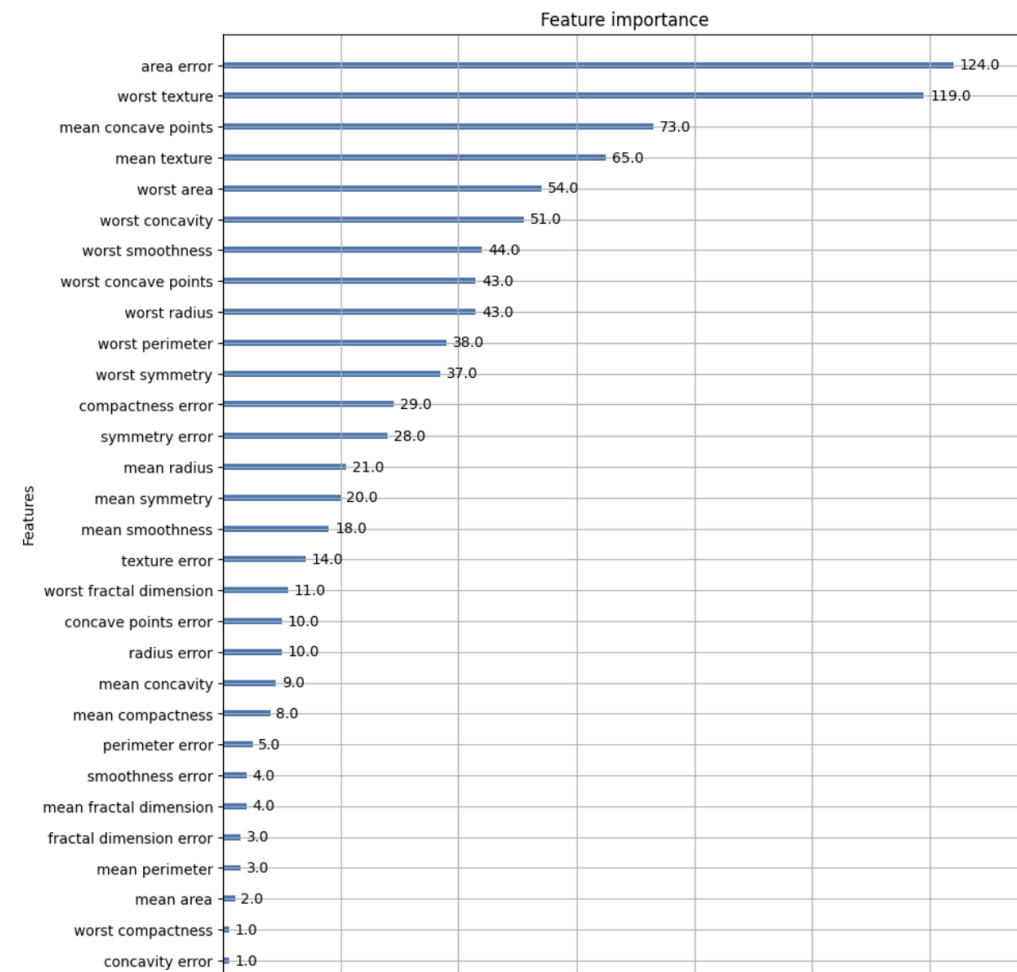
```

- 파이썬 래퍼 XGBoost는 train() 함수를 호출하여 학습 완료된 모델 객체를 반환하게 됨
- 이 모델객체는 예측 위해 predict() 메서드 이용
 - 유의점) 사이킷런의 predict() 메서드는 예측 결과 클래스 값 (0,1)을 반환하는 데 반해 xgboost의 predict()는 예측 결과값이 아닌 예측 결과를 추정할 수 있는 확률 값을 반환함

xgboost 패키지에 내장된 시각화 기능 수행

```
import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(figsize=(10, 12))
plot_importance(xgb_model, ax=ax)
plt.savefig('p239_xgb_feature_importance.tif', format='tif', dpi=300, bbox_inches='tight')
```



- xgboost의 plot importance() API는 feature의 중요도를 막대 그래프 형식으로 나타냄
- 기본 평가 지표로 f스코어를 기반으로 해당 feature 중요도 나타냄
 - f스코어는 해당 feature가 트리 분할 시 얼마나 자주 사용되었는지를 지표로 나타내는 값
- 사이킷런은 Estimator 객체의 feature_importances_ 속성을 이용해 직접 시각화 코드 작성
- xgboost 패키지는 plot_importance()를 이용해 바로 feature 중요도 시각화 가능
- plot_importance() 호출 시 파라미터로 앞에서 학습 완료된 모델 객체 및 맷플롯립의 ax 객체를 입력하기만 하면 됨
- 내장된 plot_importance() 이용 시 유의 점

- xgboost를 DataFrame이 아닌 넘파이 기반의 feature 데이터로 학습 시에는 넘파이에서 feature명을 제대로 알 수 없음
- 그러므로 Y축의 feature명을 나열 시 f0, f1과 같이 feature 순서별로 f자 뒤에 순서를 부터 feature 명 나타냄

결정 트리에서 보여준 트리 기반 규칙 구조도 xgboost에서 시각화 가능

- xgboost 모듈의 to_graphviz() API를 이용하면 주피터 노트북에 바로 규칙 트리 구조 그릴 수 있음
- 단, 결정 트리와 마찬가지로 Graphviz 프로그램과 패키지가 설치되어야 함
- xgboost.to_graphviz() 내에 파라미터로 학습이 완료된 모델 객체와 Graphviz가 참조할 파일명을 입력해주면 됨
- 파이썬 래퍼 XGBoost는 사이킷런 GridSearchCV와 유사하게 데이터 세트에 대한 교차 검증 수행
 - 후 최적 파라미터 구하는 방법 cv() API로 제공

사이킷런 래퍼 XGBoost의 개요 및 적용

- XGBoost 개발 그룹은 사이킷런 프레임워크와 연동 위해 사이킷런 전용의 XGBoost 래퍼 클래스 개발함
- 사이킷런의 기본 Estimator를 그대로 상속하여 만들
 - 다른 Estimator와 동일하게 fit()과 predict()만으로 학습과 예측 가능
 - GridSearchCV, Pipeline 등 사이킷런의 다른 유틸리티를 그대로 사용 가능
- 사이킷런 위한 래퍼 XGBoost
 - 분류를 위한 래퍼 클래스 XGBClassifier
 - 회귀를 위한 래퍼 클래스 XGBRegressor
- 파라미터 호환성 유지 위한 변경

- eta → learning_rate
- sub_sample → subsample
- lambda → reg_lambda
- alpha → reg_alpha

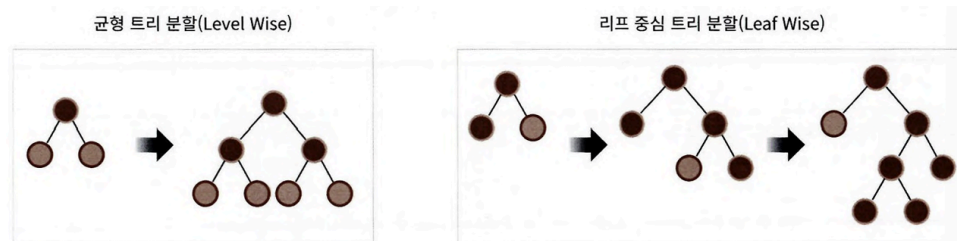
위스콘신 대학병원 유방암 데이터 세트 분류

- 위스콘신 데이터 세트의 개수 작은데 조기 중단 위해 최초 학습 데이터인 X_train을 다시 X_tr과 X_val로 분리하며 최종 학습 데이터 건수 작아지기 때문에 파이썬 래퍼 XGBoost보다 더 좋은 평가 결과 나옴
- 조기 중단 관련 파라미터를 fit()에 입력
- 조기 중단 관련 파라미터
 - 평가 지표가 향상될 수 있는 반복 횟수 정의하는 early_stopping_rounds
 - 조기 중단을 위한 평가 지표 eval_metric
 - 성능 평가 수행할 데이터 세트 eval_set
- 위스콘신 데이터 세트가 워낙 작아서 조기 중단을 위한 검증 데이터를 분리하면 검증 데이터가 없는 학습 데이터를 사용했을 때보다 성능이 저조함
- But, 조기 중단값 너무 급격하게 줄이면 예측 성능 저하될 우려 큼
- feature의 중요도를 시각화하는 모듈 plot_importance() API
 - 사이킷런 래퍼 클래스 입력해도 파이썬 래퍼 클래스 입력한 결과와 똑같이 시각화 결과 도출함

7 LightGBM

- XGBoost는 뛰어난 부스팅 알고리즘이나 학습시간 매우 길
 - XGBoost에서 GridSearchCV로 하이퍼 파라미터 튜닝을 수행하면 시간 너무 길

- 많은 파라미터 튜닝하기에 어려움 겪음
- GBM보단 빠르지만 대용량 데이터 만족할 만한 학습 성능 기대하려면 많은 CPU 코어 가진 시스템에서 높은 병렬도로 학습 진행해야 함
- LightGBM
 - XGBoost와 예측 성능은 비슷함
 - 학습 시간 훨씬 적음
 - 메모리 사용량도 훨씬 적음
 - 하지만 10,000건 이하의 데이터 세트에 적용하면 과적합 발생하기 쉬움
- 일반 GBM 계열의 트리 분할 방법과 다르게 리프 중심 트리 분할 방식
 - 기존의 대부분의 트리 기반 알고리즘은 트리 깊이를 효과적으로 줄이기 위한 균형 트리 분할 방식 사용
 - 최대한 균형 잡힌 트리 유지하면서 분할하기 때문에 트리 깊이 최소화 가능
 - 균형 잡힌 트리 생성 이유는 오버피팅에 보다 강한 구조 가질 수 있기 때문
 - 하지만 균형 맞추기 위한 시간이 필요함
- LightGBM



- 더 빠른 학습과 예측 수행 시간.
- 더 작은 메모리 사용량.
- 카테고리형 피처의 자동 변환과 최적 분할(원-핫 인코딩 등을 사용하지 않고도 카테고리형 피처를 최적으로 변환하고 이에 따른 노드 분할 수행).

- 리프 중심 트리 분할 방식은 트리 균형을 맞추지 않고 최대 손실 값을 가지는 리프 노드를 지속적으로 분할하면서 트리 깊이 깊어지고 비대칭적 규칙 트리 생성됨
- 이렇게 손실값 가지는 리프 노드를 지속적으로 분할해 생성된 규칙 트리는 학습 반복할수록 결국 균형 트리 분할 방식보다 예측 오류 손실 최소화 가능

- 사이킷런 래퍼 LightGBM 클래스는 분류 위한 LGBMClassifier 클래스와 회귀 위한 LGBMRegressor 클래스
 - 사이킷런 기반 Estimator 상속 받아 작성됐기 때문에 fit(), predict() 기반 학습 및 예측과 사이킷런 제공 유틸리티 활용 가능

LightGBM 설치

```
(base) C:\Users\inseo>pip install lightgbm==3.3.2
Collecting lightgbm==3.3.2
  Downloading lightgbm-3.3.2-py3-none-win_amd64.whl.metadata (15 kB)
Requirement already satisfied: wheel in c:\users\inseo\anaconda3\lib\site-packages (from lightgbm==3.3.2) (0.44.0)
Requirement already satisfied: numpy in c:\users\inseo\anaconda3\lib\site-packages (from lightgbm==3.3.2) (1.26.4)
Requirement already satisfied: scipy in c:\users\inseo\anaconda3\lib\site-packages (from lightgbm==3.3.2) (1.13.1)
Requirement already satisfied: scikit-learn!=0.22.0 in c:\users\inseo\anaconda3\lib\site-packages (from lightgbm==3.3.2) (1.5.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\inseo\anaconda3\lib\site-packages (from scikit-learn!=0.22.0->lightgbm==3.3.2) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\inseo\anaconda3\lib\site-packages (from scikit-learn!=0.22.0->lightgbm==3.3.2) (3.5.0)
Downloading lightgbm-3.3.2-py3-none-win_amd64.whl (1.0 MB)
1.0/1.0 MB 23.6 MB/s eta 0:00:00
Installing collected packages: lightgbm
Successfully installed lightgbm-3.3.2
```

LightGBM 하이퍼 파라미터

- LightGBM 하이퍼 파라미터는 XGBoost와 많이 유사함
 - 주의점
 - LightGBM은 Xgboost와 달리 리프 노드 계속 분할되면서 트리 깊이 깊어짐
 - 트리 특성에 맞는 하이퍼 파라미터 설정이 필요함

주요 파라미터

◦ num_iterations

num_iterations [default = 100]: 반복 수행하려는 트리의 개수를 지정합니다. 크게 지정할수록 예측 성능이 높아질 수 있으나, 너무 크게 지정하면 오히려 과적합으로 성능이 저하될 수 있습니다. 사이킷런 GBM과 XGBoost의 사이킷런 호환 클래스의 n_estimators와 같은 파라미터이므로 LightGBM의 사이킷런 호환 클래스에서는 n_estimators로 이름이 변경됩니다.

◦ learning_rate

learning_rate [default = 0.1]: 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값입니다. 일반적으로 n_estimators를 크게 하고 learning_rate를 작게 해서 예측 성능을 향상시킬 수 있으나, 마친가 지로 과적합 이슈와 학습 시간이 길어지는 부정적인 영향도 고려해야 합니다. GBM, XGBoost의 learning rate와 같은 파라미터입니다.

- max_depth

max_depth [default=-1]: 트리 기반 알고리즘의 max_depth와 같습니다. 0보다 작은 값을 지정하면 깊이에 제한이 없습니다. 지금까지 소개한 Depth wise 방식의 트리와 다르게 LightGBM은 Leaf wise 기반이므로 깊이가 상대적으로 더 깊습니다.

- min_data_in_leaf

min_data_in_leaf [default = 20]: 결정 트리의 min_samples_leaf와 같은 파라미터입니다. 하지만 사이킷런 래퍼 LightGBM 클래스인 LightGBMClassifier에서는 min_child_samples 파라미터로 이름이 변경됩니다. 최종 결정 클래스인 리프 노드가 되기 위해서 최소한으로 필요한 레코드 수이며, 과적합을 제어하기 위한 파라미터입니다.

- num_leaves

num_leaves [default = 31]: 하나의 트리가 가질 수 있는 최대 리프 개수입니다.

- boosting

boosting [default = gbdt]: 부스팅의 트리를 생성하는 알고리즘을 기술합니다.

- gbdt: 일반적인 그래디언트 부스팅 결정 트리
- rf: 랜덤 포레스트

- bagging_fraction

bagging_fraction [default = 1.0]: 트리가 커져서 과적합되는 것을 제어하기 위해서 데이터를 샘플링하는 비율을 지정합니다. 사이킷런의 GBM과 XGBClassifier의 subsample 파라미터와 동일하기에 사이킷런 래퍼 LightGBM인 LightGBMClassifier에서는 subsample로 동일하게 파라미터 이름이 변경됩니다.

- feature_fraction

feature_fraction [default = 1.0]: 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율입니다. 과적합을 막기 위해 사용됩니다. GBM의 max_features와 유사하며, XGBClassifier의 colsample_bytree와 똑같므로 LightGBMClassifier에서는 동일하게 colsample_bytree로 변경됩니다.

- lambda_l2

lambda_l2 [default=0.0]: L2 regulation 제어를 위한 값입니다. 피처 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다. XGBClassifier의 reg_lambda와 동일하므로 LightGBMClassifier에서는 reg_lambda로 변경됩니다.

- `lambda_l1`

`lambda_l1` [default = 0.0]: L1 regulation 제어를 위한 값입니다. L2와 마찬가지로 과적합 제어를 위한 것이며, XGBClassifier의 `reg_alpha`와 동일하므로 LightGBMClassifier에서는 `reg_alpha`로 변경됩니다.

- Learning Task 파라미터

- `objective`

`objective`: 최솟값을 가져야 할 손실함수를 정의합니다. Xgboost의 `objective` 파라미터와 동일합니다. 애플리케이션 유형, 즉 회귀, 다중 클래스 분류, 이진 분류인지에 따라서 `objective`인 손실함수가 지정됩니다.

하이퍼 파라미터 튜닝 방안

- `num_leaves`의 개수 중심으로 `min_child_samples(min_data_in_leaf)`, `max_depth`를 함께 조정
 - 모델 복잡도 줄이는 것이 기본 튜닝 방안
 - `num_leaves`는 개별 트리가 가질 수 있는 최대 리프 개수
 - LightGBM 모델의 복잡도를 제어하는 주요 파라미터
 - 일반적으로 `num_leaves`의 개수 높이면 정확도 높아짐, 트리 깊이 깊어지고 복잡도 커져 과적합 영향도 커짐
 - `min_data_in_leaf`
 - 사이킷 래퍼 클래스에서는 `min_child_samples`로 이름 바뀜
 - 과적합 개선 위한 중요 파라미터
 - `num_leaves`와 학습 데이터의 크기에 따라 달라지지만 보통 큰 값 설정하면 트리 깊어지는 것 방지
 - `max_depth`는 명시적으로 깊이 크기 제한
 - `num_leaves`, `min_data_in_leaf`와 결합해 과적합 개선하는 데 이용
- `learning_rate` 작게 하면서 `n_estimators` 크게 하는 것이 부스팅 계열 튜닝 가장 기본적인 튜닝 방안
 - `n_estimators` 너무 크게 하는 것은 과적합으로 오히려 성능 저하될 수 있음

파이썬 래퍼 LightGBM과 사이킷런 래퍼 XGBoost, LightGBM 하이퍼 파라미터 비교

- LightGBM은 사이킷런 호환하기 위해 분류 위한 LGBMClassifier와 회귀를 위한 LGBMRegressor 클래스를 래퍼 클래스로 생성
- 사이킷런 래퍼 클래스를 제공한 XGBoost는 사이킷런 하이퍼 파라미터 명명 규칙에 따라 자신의 하이퍼 파라미터 변경

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

LightGBM 적용 - 위스콘신 유방암 예측

```

from lightgbm import LGBMClassifier, early_stopping
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import pandas as pd

# 데이터 로딩
dataset = load_breast_cancer()
cancer_df = pd.DataFrame(data=dataset.data, columns=dataset.feature_names)
cancer_df['target'] = dataset.target

X = cancer_df.iloc[:, :-1]
y = cancer_df.iloc[:, -1]

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.1, random_state=42)

# 모델 초기화
lgbm = LGBMClassifier(n_estimators=400, learning_rate=0.05)

# ✅ 조기 종료 콜백 설정
early_stopping_cb = early_stopping(stopping_rounds=50, verbose=True)

# 모델 학습
lgbm.fit(
    X_tr, y_tr,
    eval_set=[(X_val, y_val)],
    eval_metric='binary_logloss',
    callbacks=[early_stopping_cb]
)

# 예측
preds = lgbm.predict(X_test)
pred_proba = lgbm.predict_proba(X_test)[:, 1]

```

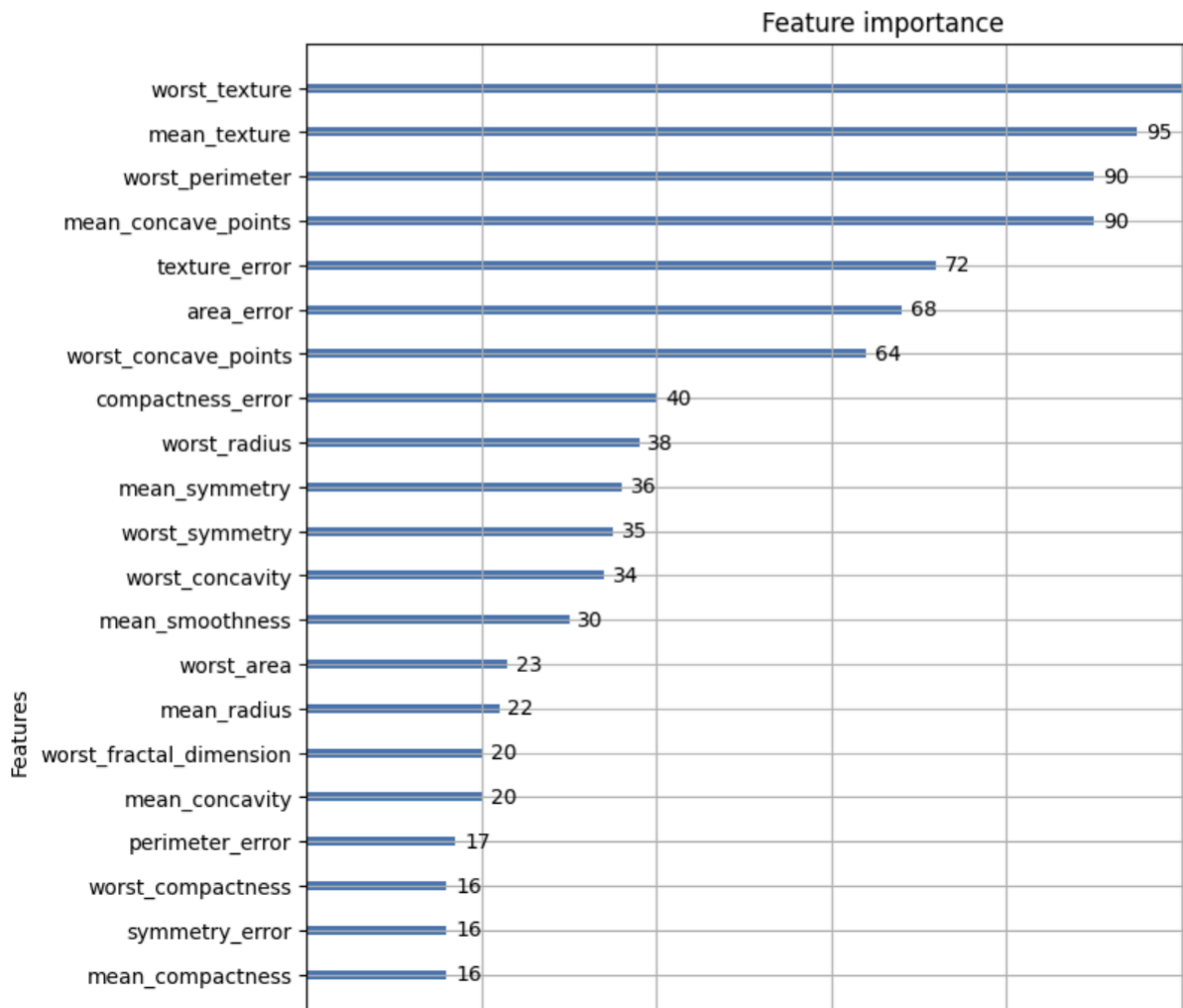
- 조기 중단으로 11번 반복까지만 수행, 학습 종료
- plot_importance() feature 중요도 시각화할 수 있는 내장 API 제공

```

from lightgbm import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(figsize=(10, 12))
plot_importance(lgbm_wrapper, ax=ax)
plt.savefig('lightgbm_feature_importance.tif', format='tif', dpi=300, bbox_inches='tight')

```



8 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

- Grid Search 방식은 튜닝해야 할 하이퍼 파라미터 개수가 많을 경우 최적화 수행 시간이 오래 걸림
 - 여기에 개별 하이퍼 파라미터 값의 범위가 넓거나 학습 데이터가 대용량인 경우 최적화 시간 더 늘어남

- XGBoost나 LightGBM은 성능이 뛰어나
 - 다만 하이퍼 파라미터 개수가 다른 알고리즘에 비해 많음
 - 실무 대용량 학습 데이터에 Grid Search 방식으로 최적 하이퍼 파라미터 찾으려면 많은 시간 소모할 수 있음

베이지안 최적화 개요

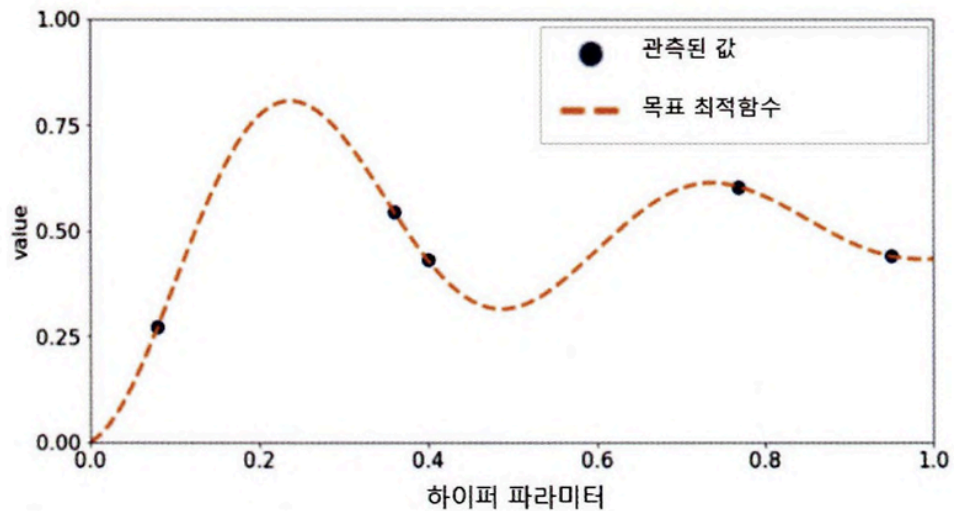
베이지안 최적화

- 목적 함수 식을 제대로 알 수 없는 블랙 박스 형태의 함수에서 최대 또는 최소 함수 반환 값을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식
- 특정 함수 식을 알고 있다고 가정
 - 함수 $f(x,y) = 2x - 3y$
 - $f(x,y)$ 의 반환 값을 최대/최소로 하는 x,y 값을 찾는것
 - x 는 크면 클수록(x 값의 범위가 0~1000이라면 1000), y 는 0일 경우 가장 최대의 값을반환
- 함수 식 자체를 모르거나 함수 식이 복잡/ 입력값 개수 많/ 범위 넓은 경우 베이지안 최적화 이용하면 쉽게 최적 입력값 찾을 수 있음
- 베이지안 확률
 - 새로운 사건의 관측이나 새로운 샘플 데이터를 기반으로 사후 확률을 개선해나감
- 베이지안 최적화
 - 새로운 데이터를 입력 받았을 때 최적 함수를 예측하는 사후 모델 개선하며 최적 함수 모델 만들어냄
 - 두 가지 중요 요소
 - 대체 모델 (Surrogate Model)
 - 획득 함수로부터 최적 함수를 예측할 수 있는 입력값을 추천 받은 뒤
 - 이를 기반으로 최적 함수 모델 개선해나감
 - 획득 함수(Acquisition Function)
 - 개선된 대체 모델 기반으로 최적 입력값 계산함

베이지안 최적화 단계

- Step1

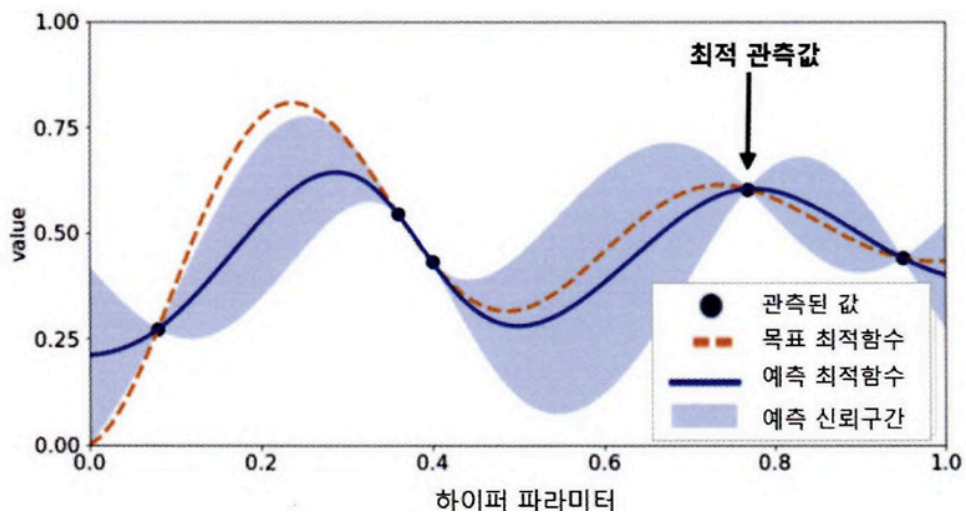
- 최초에는 랜덤하게 하이퍼 파라미터들 샘플링, 성능 결과 관측



- 검은원 : 특정 하이퍼 파라미터가 입력되어 관측된 성능 지표 결과값
- 주황 사선 : 찾아야 할 목표 최적 함수

- Step2

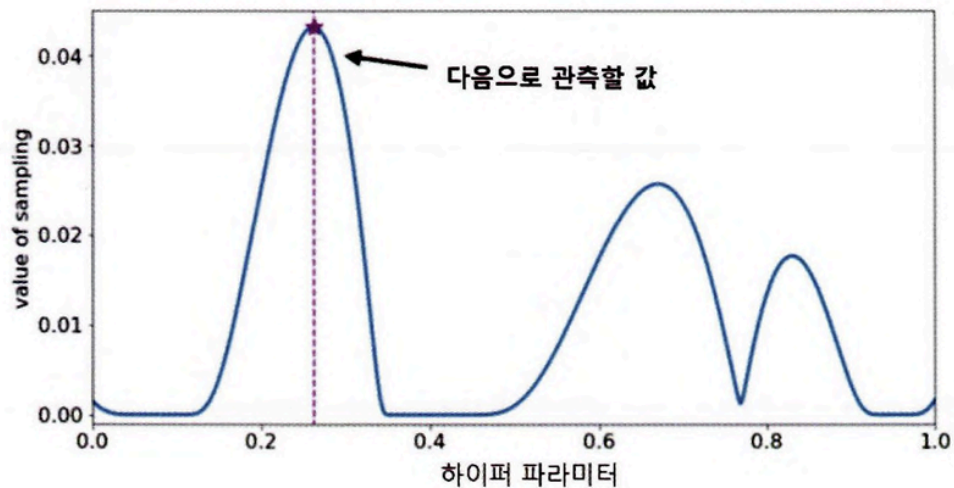
- 관측된 값 기반, 대체 모델은 최적 함수 추정



- 파란 실선: 대체 모델이 추정한 최적 함수

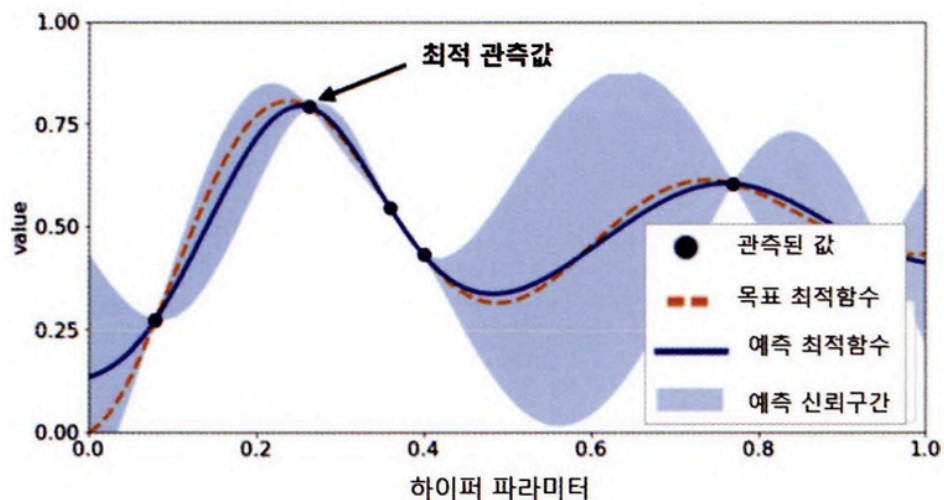
- 옆은 파란색 영역: 예측된 함수의 신뢰 구간
 - 추정된 함수의 결괏값 오류 편차 의미하며 추정 함수의 불확실성 나타냄
- 최적 관측값은 y축 value에서 가장 높은 값 가질 때의 하이퍼 파라미터

• Step3



- 추정된 최적 함수를 기반으로 획득 함수는 다음 관측할 하이퍼 파라미터 값 계산
- 획득 함수는 이전 최적 관측값보다 더 큰 최댓값 가질 가능성 높은 지점 찾아서 다음에 관측할 하이퍼 파라미터 대체 모델에 전달

• Step4



- 획득 함수로부터 전달된 하이퍼 파라미터 수행하여 관측된 값을 기반으로 대체 모델은 갱신되어 다시 최적 함수를 예측 추정

HyperOpt 사용하기

HyperOpt 활용 주요 로직

- 입력 변수명과 입력값의 검색 공간 설정
- 목적 함수의 설정
- 목적 함수의 반환 최솟값을 가지는 최적 입력값을 유추하는 것

다른 패키지과 다르게 목적 함수 반환 값의 최댓값이 아닌 최솟값을 가지는 최적 입력 값을 유추함

입력 변수명, 입력값의 검색 공간 설정

```
from hyperopt import hp
search_space = {'x': hp.quniform('x', -10, 10, 1), 'y': hp.quniform('y', -15, 15, 1) }
```

- 입력 변수명과 입력값 검색 공간은 파이썬 딕셔너리 형태로 설정되어야 함
- 키(key)값으로 입력 변수명, 밸류(value) 값으로 해당 입력 변수의 검색 공간이 주어짐
- hp 모듈은 입력값의 검색 공간을 다양하게 설정
- hp.quniform('x', -10, 10, 1)처럼 설정하면 입력 변수 x는 -10부터 10까지 1의 간격을 가지는 값
 - [-10, -9, -8, ..., 8, 9, 10]과 같은 값을 가짐
 - 다만 값들은 순차적으로 입력되지 않음

입력값의 검색 공간 제공하는 대표적 함수들

- low: 최솟값
- high: 최댓값
- q: 간격

- `hp.quniform(label, low, high, q)`: label로 지정된 입력값 변수 검색 공간을 최소값 low에서 최대값 high까지 q의 간격을 가지고 설정.
- `hp.uniform(label, low, high)`: 최소값 low에서 최대값 high까지 정규 분포 형태의 검색 공간 설정
- `hp.randint(label, upper)`: 0부터 최대값 upper까지 random한 정숫값으로 검색 공간 설정.
- `hp.loguniform(label, low, high)`: `exp(uniform(low, high))` 값을 반환하며, 반환 값의 log 변환 된 값은 정규 분포 형태를 가지는 검색 공간 설정.
- `hp.choice(label, options)`: 검색 값이 문자열 또는 문자열과 숫자값이 섞여 있을 경우 설정. Options는 리스트나 튜플 형태로 제공되며 `hp.choice('tree_criterion', ['gini', 'entropy'])`과 같이 설정하면 입력 변수 `tree_criterion`의 값을 'gini'와 'entropy'로 설정하여 입력함.

목적 함수 생성

```
from hyperopt import STATUS_OK

def objective_func(search_space):
    x = search_space['x']
    y = search_space['y']
    retval = x**2 - 20*y

    return retval
```

- 반드시 변숫값과 검색 공간을 가지는 딕셔너리를 인자로 받아야 함
- 특정 값 반환하는 구조로 만들어져야 함

목적 함수의 반환값 최소 되는 최적 입력값 찾기

HyperOpt는 `fmin()` 함수 제공

- fn: 위에서 생성한 objective_func와 같은 목적 함수입니다.
- space: 위에서 생성한 search_space와 같은 검색 공간 딕셔너리입니다.
- algo: 베이지안 최적화 적용 알고리즘 입니다. 기본적으로 tpe.suggest이며 이는 HyperOpt의 기본 최적화 알고리즘인 TPE(Tree of Parzen Estimator)를 의미합니다.
- max_evals: 최적 입력값을 찾기 위한 입력값 시도 횟수입니다.
- trials: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는 데 사용됩니다. Trials 클래스를 객체로 생성한 변수명을 입력합니다.
- rstate: fmin()을 수행할 때마다 동일한 결괏값을 가질 수 있도록 설정하는 랜덤 시드(seed) 값입니다.

search_space에서 목적 함수 object_func의 최솟값 반환하는 최적입력 변수값 찾기

- HyperOpt의 fmin() 함수를 호출하되 먼저 5번의 입력값 시도로 찾아낼 수 있도록고 max_evals 인자 값으로 5 설정
- 목적 함수인 fn 인자로는 objective_func
- 검색 공간 인자인 space에는 search_space
- 최적화 적용 알고리즘 algo 인자는 기본값인 tpe.suggest
- trias 인자값으로는 Trial()객체
- rstate는 임의의 랜덤 시드값 입력

```
from hyperopt import fmin, tpe, Trials
import numpy as np

trial_val = Trials()

best_01 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest, max_evals=5,
               , trials=trial_val, rstate=np.random.default_rng(seed=0))
print('best:', best_01)
```

```
100%|██████████| 5/5 [00:00<00:00, 673.52trial/s, best loss: -224.0]
best: {'x': np.float64(-4.0), 'y': np.float64(12.0)}
```

- 입력 변수 x의 공간-10 ~ 10, y의 공간 -15 ~ 15에서 목적 함수의 반환값을 $x^2 - 2 \cdot y$ 로 설정
- x는 0에 가까울수록 y는 15에 가까울수록반환값이 최소로 근사

```

trial_val = Trials()

best_02 = fmin(fn=objective_func, space=search_space, algo=tpe.suggest, max_evals=20
               , trials=trial_val, rststate=np.random.default_rng(seed=0))
print('best:', best_02)

```

100%|██████████| 20/20 [00:00<00:00, 567.33trial/s, best loss: -296.0]
best: {'x': np.float64(2.0), 'y': np.float64(15.0)}

- max_evals를 20회로 늘리면 x는 2로, y는 15로 목적 함수의 최적 최솟값 근사할 수 있는 결과 도출

261p

Trials 객체의 vals 속성

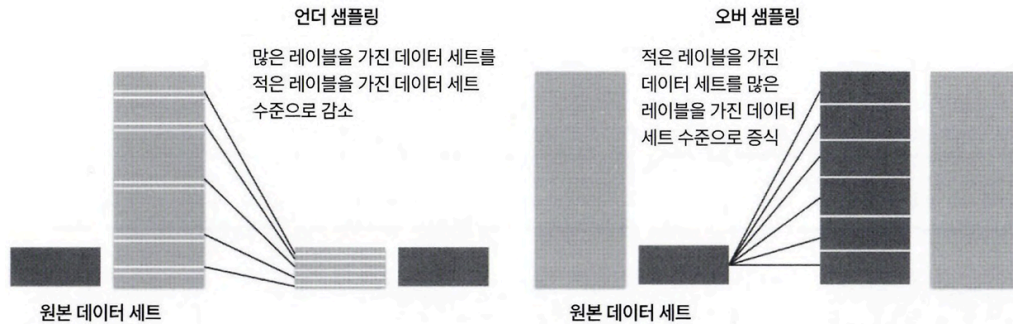
- 딕셔너리 형태로 값 가짐
- fmin() 함수 수행 시마다 입력되는 입력 변수값들 가진
 - {'입력변수명': 개별 수행 시마다 입력된 값의 리스트}의 형태

results와 vals속성값들을 DataFrame으로 만들어서 더 직관적으로 값 확인

10분류 실습 - 캐글 신용카드 사기 검출

언더 샘플링과 오버 샘플링의 이해

- 레이블이 불균형한 분포 가지는 데이터 세트 학습시킬 때 예측 성능 문제 발생가능
 - 이상 레이블을 가지는 데이터 건수가 정상 레이블을 가진 데이터 건수에 비해 너무 적기 때문
- 지도 학습에서 극도로 불균형한 레이블 값 분포로 인한 문제점 해결하기 위해 적절한 학습 데이터 확보
 - 오버 샘플링(Oversampling)
 - 예측 성능 상 조금 유리한 경우가 많아 상대적으로 더 많이 사용됨
 - 언더 샘플링(Undersampling)

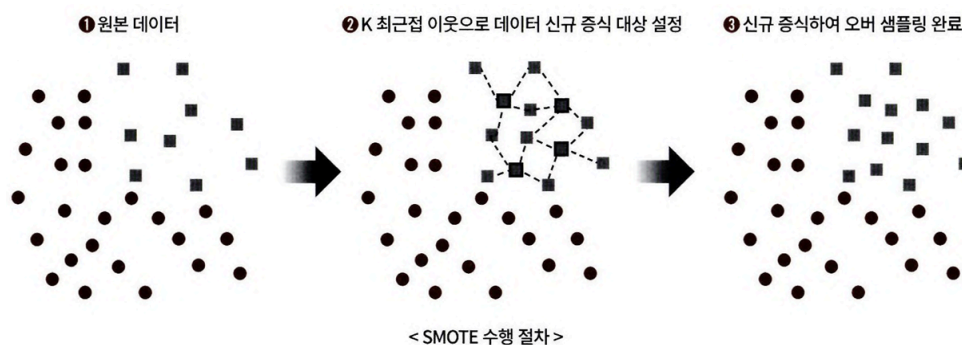


언더 샘플링

- 많은 데이터 세트를 적은 데이터 세트 수준으로 감소시키는 방식
- 정상 레이블 가진 데이터가 10,000건, 이상 레이블을 가진 데이터가 100건 있으면 정상 레이블 데이터를 100건으로 줄여버림
- 정상 레이블 데이터를 이상 레이블 데이터 수준으로 줄여 버린 상태에서 학습을 수행하면 과도하게 정상 레이블로 학습/예측하는 부작용을 개성할 수 있음
- 하지만 너무 많은 정상 레이블 데이터를 감소시켜서 정상 레이블의 경우 제대로 된 학습을 수행할 수 없는 문제가 발생할 수도 있으므로 유의 필요

오버 샘플링

- 이상 데이터와 같이 적은 데이터 세트를 증식하여 학습을 위한 충분한 데이터를 확보하는 방법
- 동일 데이터를 단순히 증식하는 방법은 과적합이 되기 때문에 의미가 없으므로 원본 데이터의 feature 값들을 아주 약간만 변경하여 증식함
- 대표적으로 SMOTE 방법이 있음
 - (Synthetic Minority Over-sampling Technique)



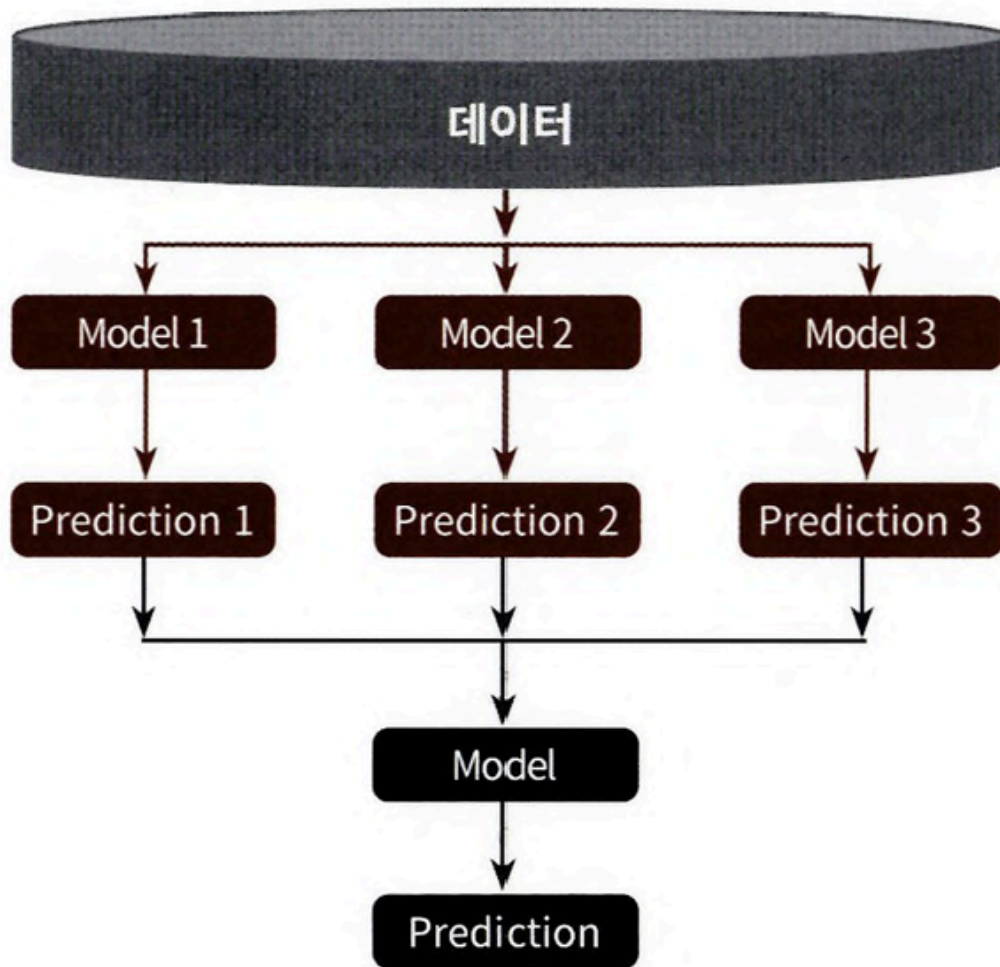
- 적은 데이터 세트에 있는 개별 데이터들의 K 최근접 이웃을 차장 이 데이터와 K 개 이웃들의 차이를 일정 값으로 만들어서 기존 데이터와 약간 차이가 나는 새로운 데이터들을 생성하는 방식
- SMOTE 구현한 대표적 파이썬 패키지 imbalanced-learn

11 스택킹 앙상블

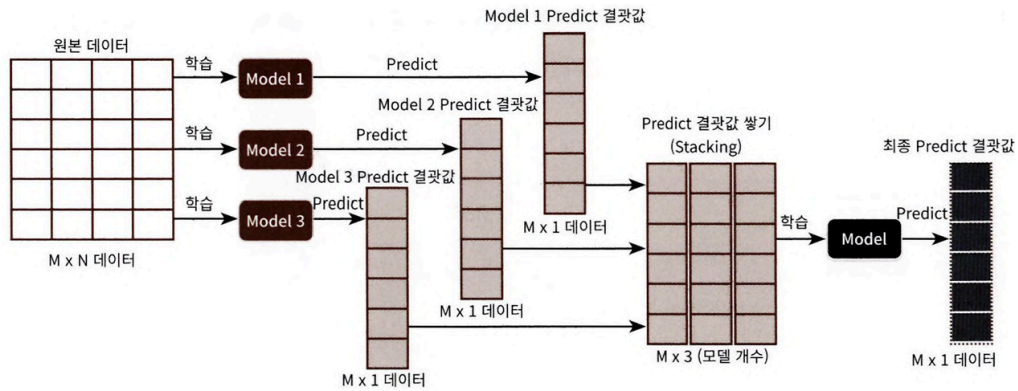
스택킹

- 개별적인 여러 알고리즘을 서로 결합해 예측 결과를 도출함
 - 배깅, 부스팅과의 공통점
- 개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측 수행
 - 배깅, 부스팅과의 차이점
- 즉, 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종 학습을 수행하고 테스트 데이터를 기반으로 다시 최종 예측 수행
 - 메타 모델
 - 개별 모델의 예측된 데이터 세트를 다시 기반으로 하여 학습하고 예측하는 방식

스택킹 모델



- 두 종류의 모델 필요함
 - 개별적 기반 모델
 - 최종 메타 모델
 - 개별적 기반 모델의 예측 데이터를 학습 데이터로 만들어서 학습함
- 여러 개별 모델의 예측 데이터를 각각 스택킹 형태로 결합해 최종 메타 모델의 학습용 feature 데이터 세트와 테스트용 feature 데이터 세트를 만듦
- 스택킹 적용할 때는 많은 개별 모델이 필요함
 - 2-3개 개별 모델만을 결합해서는 쉽게 예측 성능 향상시킬 수 없음
 - 스택킹 적용한다고 해서 반드시 성능 향상 되리라는 보장도 없음
 - 성능 비슷한 모델을 결합해 좀 더 나은 성능 향상 도출하기 위해 적용됨



- M개의 로우, N개의 feature 칼럼을 가진 데이터 세트에 스택킹 앙상블 적용한다고 가정
- 학습에 사용될 ML 알고리즘 모델 3개
- 모델별로 각각 학습시킨 뒤 예측 수행하면 각각 M개의 로우를 가진 1개의 레이블 값도 출력
- 모델 별로 도출된 예측 레이블 값 다시 스택킹하여 새로운 데이터 세트 만들고 스택킹된 데이터 세트에 대해 최종 모델 적용, 최종 예측

기본 스택킹 모델

CV 세트 기반의 스택킹

CV 세트 기반의 스택킹 모델

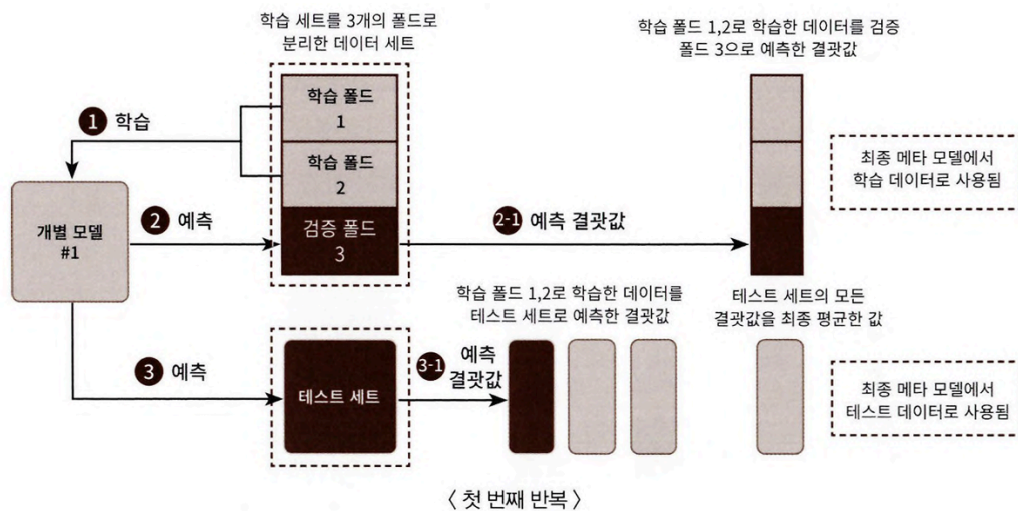
- 과적합 개선하기 위해 최종 메타 모델 위한 데이터 세트 만들 때 교차 검증 기반으로 예측된 결과 데이터 세트 이용
- Step1
 - 각 모델별로 원본 학습/테스트 데이터를 예측한 결과 값을 기반으로 메타 모델을 위한 학습용/테스트용 데이터를 생성합니다
- Step2
 - 1단계에서 개별 모델들이 생성한 학습용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 학습할 최종 학습용 데이터 세트를 생성

- 마찬가지로 각 모델들이 생성한 테스트용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 예측할 최종 테스트 데이터 세트를 생성
- 메타 모델은 최종적으로 생성된 학습 데이터 세트와 원본 학습 데이터의 레이블 데이터를 기반으로 학습한 뒤
 - 최종적으로 생성된 테스트 데이터 세트 예측
 - 원본 테스트 데이터의 레이블 데이터를 기반으로 평가

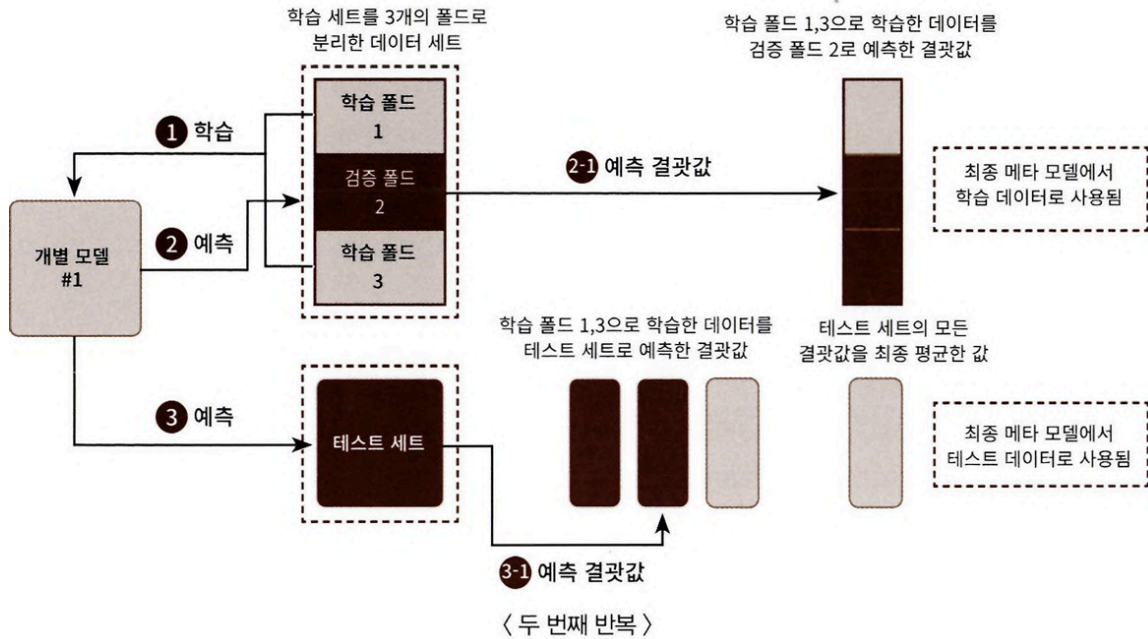
- 학습용 데이터를 N개의 Fold로 나눔
- 그림에서는 3개의 Fold
- 3개의 Fold 세트이므로 3번의 유사한 반복 작업 수행
- 마지막 3번째 반복에서 개별 모델의 예측값으로 학습 데이터와 테스트 데이터를 생성

Step 1

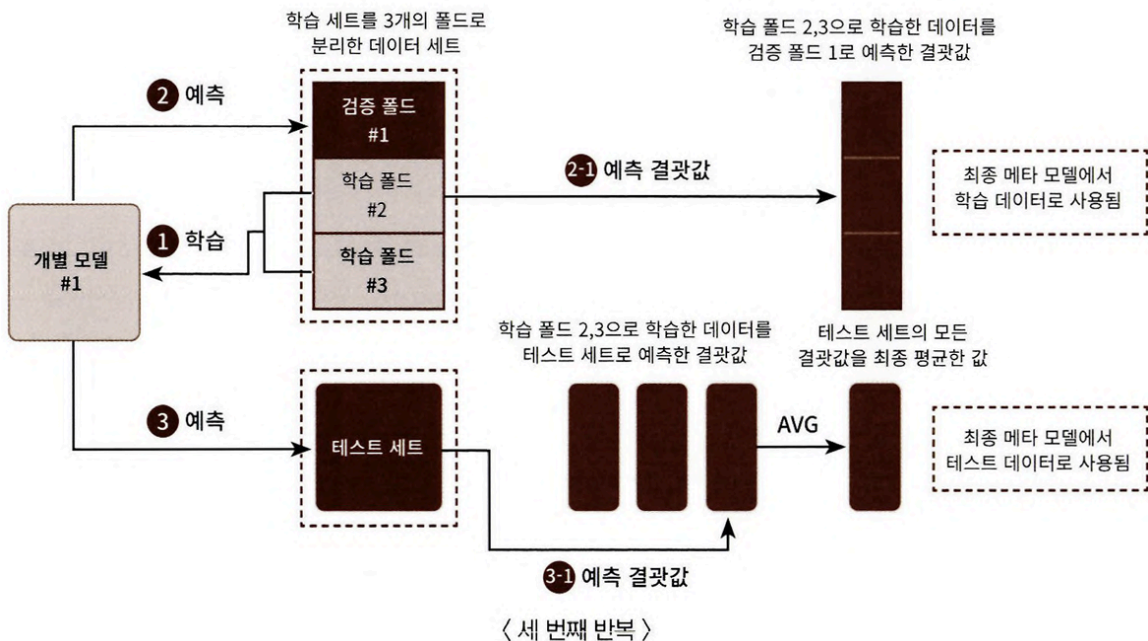
1. 학습용 데이터를 3개의 Fold로 나누되, 2개의 Fold는 학습을 위한 데이터 Fold, 나머지 1개의 Fold는 검증을 위한 데이터 Fold



1. 두 개의 Fold로 나뉜 학습 데이터를 기반으로 개별 모델을 학습시킴
2. 학습된 개별 모델은 검증 Fold 1개 데이터로 예측하고 그 결과 저장



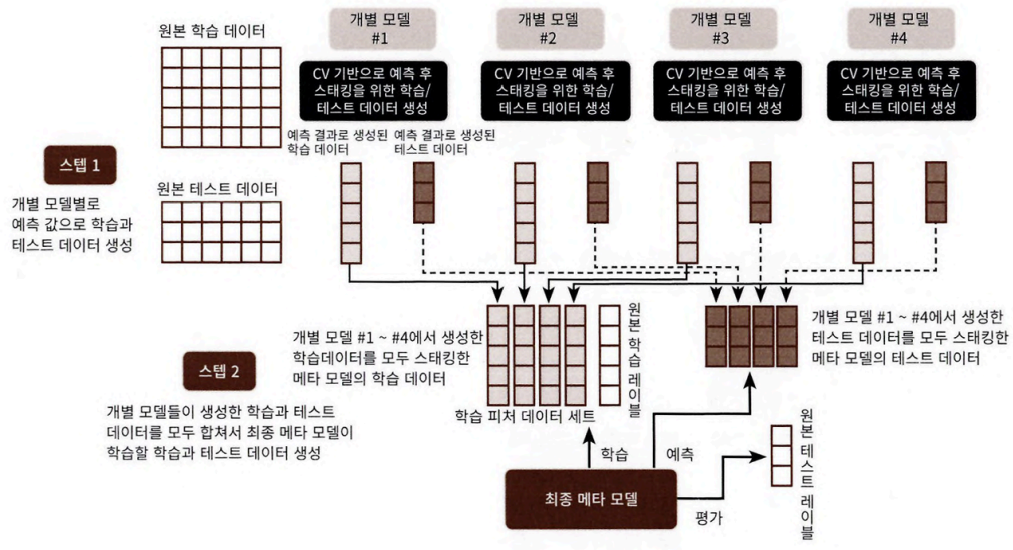
1. 이러한 로직 3번 반복하면서 학습 데이터와 검증 데이터 세트를 변경해가면서 학습 후 예측 결과를 별도 저장
2. 이렇게 만들어진 예측 데이터는 메타 모델을 학습시키는 학습 데이터로 사용
3. 2개의 학습 Fold 데이터로 학습된 개별 모델은 원본 테스트 데이터를 예측하여 예측값을 생성



1. 마찬가지로 이러한 로직을 3번 반복하면서 이 예측값의 평균으로 최종 결과값을 생성

2. 이를 메타 모델을 위한 테스트 데이터로 사용

Step 2



- 각 모델들이 스텝1로 생성한 학습과 테스트 데이터를 모두 합쳐 최종적으로 메타 모델이 사용할 학습 데이터와 테스트 데이터 생성
- 메타 모델이 사용할 최종 학습 데이터와 원본 데이터의 레이블 데이터를 합쳐 메타 모델 학습 후 최종 테스트 데이터로 예측 수행 한 뒤
 - 최종 예측 결과를 원본 테스트 데이터의 레이블 데이터와 비교해 평가